Report On
Library Management System

Division: I
Under the Guidance of Dr. Bhagyashri S. Deshpande

Soham Gonge - RBT25CB063
Aayush Kumar - RBT25CB045
Ritesh Buchade - RBT25CB045

1) Existing System & Need for System

Traditional library systems rely on manual record-keeping using registers or spreadsheets. This results in inefficiencies, data inconsistency, and difficulty in tracking records. Need: A digital system ensures faster operations, accurate tracking, and automation of tasks like issue/return and fine calculation.

2) Scope of Work

The project includes: • Admin authentication • Book management (add/remove/search) • Member management • Issue/Return tracking • Fine calculation (Rs.2/day) • File-based data storage Excludes GUI and database systems.

3) Proposed System – Module Description

User Module: Base class for users Admin Module: Handles authentication and admin controls Member Module: Handles borrowing and fine calculation Book Module: Stores book details Library Module: Core system handling all operations

4) Objective of System

• Automate library operations • Reduce manual errors • Enable quick search and tracking • Maintain persistent data • Demonstrate OOP concepts in C++

5) Operating Environment

Software: C++17, g++ OS: macOS / Windows / Linux Tools: VS Code, STL libraries Hardware: Processor: i3+ RAM: 2GB+ Storage: 50MB

6) Code of Project

user.h

```
#ifndef USER_H
#define USER_H

#include <iostream>
using namespace std;

class user
{
protected:
    string name;
    string contactNumber;

public:
    user();
    user(string pname, string pcontactNumber);

    void set_name(string pname);
    void set_contactNumber(string pcontactNumber);

    string get_name() const;
    string get_contactNumber() const;
};

#endif
```

admin.h

```
#ifndef ADMIN_H
#define ADMIN_H

#include<iostream>
#include<cstring>
#include"user.h"
using namespace std;

class admin: public user{
    private :
    int adminId;
    string password;
    bool isAuthenticated = false;

    public :
        // Constructors
        admin();
        admin(string pname, string pcontactNumber, int padminId, string ppassword);
        // getter
        int get_adminId();
        bool get_isAuthenticated();
        bool authenticate(string inputPassword);
};

#endif
```

member.h

```
#ifndef MEMBER_H
#define MEMBER_H

#include <iostream>
#include <vector>
#include "user.h"
using namespace std;

class member : public user
{
private:
    int memberId;
    vector<string> borrowedBooks; // stores ISBNs as strings

public:
    member();
    member(string pname, string pcontactNumber, int pmemberId);

    // getters
    int get_memberId() const;
    vector<string> get_borrowedBooks() const;

    // setters
    void set_memberId(int pmemberId);
    void addBorrowedBook(string isbn);
    void removeBorrowedBook(string isbn);
    float calculateFine(int pdaysOverDue); // Rs.2 per day
};

#endif
```

library.h

```
#ifndef LIBRARY_H
#define LIBRARY_H

#include<iostream>
#include "book.h"
#include "member.h"
using namespace std;

class library
{
private:
    vector<book> Books;
    vector<member> Members;

public:
//    Books Management
    void addBook(const book& b); // push_back onto books vector
    void removeBook(string isbn); // find by isbn, erase form vector
    void updateBook(string isbn); // find book, call setters on it
    void searchBook(string query); // search by title or aurthor, print matches
    void displayAllBooks(); // loop through books vectors, print each
    void displayAvailableBooks(); // loop, print only where availableCopies>0
    void displayBorrowedBooks(); // loop, print only where availableCopies < totalCopies
    void displayOverdueBooks(); // requires due date tracking
    void getMostBorrowedBooks(); // sort or count borrow frequency


    // Member Management
    void registerMember(member m);
    void removeMember(int memberId);
    bool authenticateMember(int memberId, string contact);

    // Borrow & Return
    void issueBook(int memberId, string isbn); // find member + book, decrement availableCopies, cal
    void returnBook(int memberId, string isbn); // extend due date if no reservation
    void payFine(int memberId); // call member.calculateFine(), record payment

    // Persistence

    void saveBooksToFile(); // write books vector to data/books.txt
    void loadBooksFromFile(); // read data/books.txt, reconstruct books vector
    void saveMemberToFile(); // write member vector to data/members.txt
    void loadMembersFromFile(); // read data/members.txt, reconstruct members vector
};


#endif
```

book.h

```
#ifndef BOOK_H
#define BOOK_H

#include <iostream>
#include <cstring>
using namespace std;

class book
{
private:
    string title;
    string author;
    string genre;
    string isbn;
    int totalCopies;
    int availableCopies;

public:
    // Constructors
    book() = default;
    book(string ptitle, string pauthor, string pgenre, string pisbn, int ptcopies, int pacopies);

    // Getters
    string get_title() const ;
    string get_author() const ;
    string get_genre() const ;
    string get_isbn() const ;
    int get_tcopies() const ;
    int get_acopies() const ;

    // Setters
    void set_title(string ptitle);
    void set_author(string pauthor);
    void set_genre(string pgenre);
    void set_isbn(string pisbn);
    void set_tcopies(int ptcopies);
    void set_acopies(int pacopies);

    // operator overloading
    friend ostream &operator<<(ostream &os, const book &b);
};
#endif
```

admin.cpp

```
#include <iostream>
#include <cstring>
#include "admin.h"
#include "user.h"
using namespace std;

admin::admin()
{
}
admin::admin(string pname, string pcontactNumber, int padminId, string ppassword)
{
    name = pname;
    contactNumber = pcontactNumber;
    adminId = padminId;
    password = ppassword;
}
// getter
int admin::get_adminId()
{
    return adminId;
}
bool admin::get_isAuthenticated()
{
    return isAuthenticated;
}
bool admin::authenticate(string inputPassword)
{
    if (password == inputPassword)
    {
        isAuthenticated = true;
        return true;
    }
    else{
        return false;
    }

}
```

user.cpp

```
#include <iostream>
#include "user.h"
using namespace std;

user::user() {}

user::user(string pname, string pcontactNumber)
{
    name = pname;
    contactNumber = pcontactNumber;
}

void user::set_name(string pname)
{
    name = pname;
}

void user::set_contactNumber(string pcontactNumber)
{
    contactNumber = pcontactNumber;
}

string user::get_name() const
{
    return name;
}

string user::get_contactNumber() const
{
    return contactNumber;
}
```

library.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>
#include "user.h"
#include "member.h"
#include "admin.h"
#include "library.h"
#include "book.h"
using namespace std;

// ═══════════════════════════════════════════════════
//  HELPER — split a line by a delimiter
//  replaces sstream entirely
// ═══════════════════════════════════════════════════
vector<string> splitLine(const string &line, char delim)
{
    vector<string> fields;
    string current;
    for (char c : line)
    {
        if (c == delim)
        {
            fields.push_back(current);
            current = "";
        }
        else
        {
            current += c;
        }
    }
    fields.push_back(current); // last field
    return fields;
}

// ═══════════════════════════════════════════════════
//  BOOK MANAGEMENT
// ═══════════════════════════════════════════════════

void library::addBook(const book &b)
{
    Books.push_back(b);
    cout << "Book \"" << b.get_title() << "\" added successfully." << endl;
}

void library::removeBook(string isbn)
{
    int idx = -1;
    for (int i = 0; i < (int)Books.size(); i++)
    {
        if (Books[i].get_isbn() == isbn)
        {
            idx = i;
            break;
        }
    }
    if (idx == -1)
    {
        cout << "Book with ISBN " << isbn << " not found." << endl;
        return;
    }

    cout << "Book \"" << Books[idx].get_title() << "\" removed successfully." << endl;
    Books.erase(Books.begin() + idx);
}

void library::updateBook(string isbn)
{
    for (int i = 0; i < (int)Books.size(); i++)
    {
        if (Books[i].get_isbn() == isbn)
        {
```

```
            string title, author, genre;
            int tcopies, acopies;

            cout << "Enter new title (current: " << Books[i].get_title() << "): ";
            cin.ignore();
            getline(cin, title);
            Books[i].set_title(title);

            cout << "Enter new author (current: " << Books[i].get_author() << "): ";
            getline(cin, author);
            Books[i].set_author(author);

            cout << "Enter new genre (current: " << Books[i].get_genre() << "): ";
            getline(cin, genre);
            Books[i].set_genre(genre);

            cout << "Enter total copies (current: " << Books[i].get_tcopies() << "): ";
            cin >> tcopies;
            Books[i].set_tcopies(tcopies);

            cout << "Enter available copies (current: " << Books[i].get_acopies() << "): ";
            cin >> acopies;
            Books[i].set_acopies(acopies);

            cout << "Book updated successfully." << endl;
            return;
        }
    }
    cout << "Book with ISBN " << isbn << " not found." << endl;
}

void library::searchBook(string query)
{
    string lowerQuery = query;
    transform(lowerQuery.begin(), lowerQuery.end(), lowerQuery.begin(),
              [](unsigned char c)
              { return tolower(c); });

    cout << "\n--- Search Results for \"" << query << "\" ---" << endl;
    bool found = false;
    for (auto &b : Books)
    {
        string lowerTitle = b.get_title();
        string lowerAuthor = b.get_author();
        transform(lowerTitle.begin(), lowerTitle.end(), lowerTitle.begin(),
                  [](unsigned char c)
                  { return tolower(c); }); // lamda function
        transform(lowerAuthor.begin(), lowerAuthor.end(), lowerAuthor.begin(),
                  [](unsigned char c)
                  { return tolower(c); });

        if (lowerTitle.find(lowerQuery) != string::npos ||
            lowerAuthor.find(lowerQuery) != string::npos)
        {
            cout << b << endl;
            found = true;
        }
    }
    if (!found)
        cout << "No books found matching: " << query << endl;
}

void library::displayAllBooks()
{
    if (Books.empty())
    {
        cout << "No books in the library." << endl;
        return;
    }
    cout << "\n--- All Books ---" << endl;
    cout << "Title | Author | Genre | ISBN | Available/Total" << endl;
    cout << string(60, '-') << endl;
    for (auto &b : Books)
        cout << b << endl;
}

void library::displayAvailableBooks()
```

```
{
    cout << "\n--- Available Books ---" << endl;
    bool found = false;
    for (auto &b : Books)
        if (b.get_acopies() > 0)
        {
            cout << b << endl;
            found = true;
        }
    if (!found)
        cout << "No available books at the moment." << endl;
}

void library::displayBorrowedBooks()
{
    cout << "\n--- Currently Borrowed Books ---" << endl;
    bool found = false;
    for (auto &b : Books)
        if (b.get_acopies() < b.get_tcopies())
        {
            cout << b << endl;
            found = true;
        }
    if (!found)
        cout << "No books are currently borrowed." << endl;
}

void library::displayOverdueBooks()
{
    cout << "\n--- Overdue Books ---" << endl;
    ifstream fin("transaction.txt");
    if (!fin.is_open())
    {
        cout << "Could not open transaction file." << endl;
        return;
    }

    bool found = false;
    string line;
    // Format: memberId|isbn|issueDate|dueDate|returned(0/1)
    while (getline(fin, line))
    {
        if (line.empty())
            continue;
        vector<string> f = splitLine(line, '|');
        if (f.size() >= 5 && f[4] == "0")
        {
            cout << "Member ID: " << f[0] << " | ISBN: " << f[1]
                 << " | Due: " << f[3] << " (not returned)" << endl;
            found = true;
        }
    }
    fin.close();
    if (!found)
        cout << "No overdue books found." << endl;
}

void library::getMostBorrowedBooks()
{
    // reads transaction.txt and counts how many times each isbn appears
    ifstream fin("transaction.txt");
    if (!fin.is_open())
    {
        cout << "Could not open transaction file." << endl;
        return;
    }

    vector<string> isbnList;
    vector<int> countList;

    string line;
    while (getline(fin, line))
    {
        if (line.empty())
            continue;
        vector<string> f = splitLine(line, '|');
        if (f.size() < 2)
```

```cpp
            continue;

        string isbn = f[1];
        bool found = false;
        for (int i = 0; i < (int)isbnList.size(); i++)
        {
            if (isbnList[i] == isbn)
            {
                countList[i]++;
                found = true;
                break;
            }
        }
        if (!found)
        {
            isbnList.push_back(isbn);
            countList.push_back(1);
        }
    }
    fin.close();

    if (isbnList.empty())
    {
        cout << "No borrow history available." << endl;
        return;
    }

    // simple selection sort by count descending
    for (int i = 0; i < (int)isbnList.size() - 1; i++)
        for (int j = i + 1; j < (int)isbnList.size(); j++)
            if (countList[j] > countList[i])
            {
                swap(countList[i], countList[j]);
                swap(isbnList[i], isbnList[j]);
            }

    cout << "\n--- Most Borrowed Books ---" << endl;
    for (int i = 0; i < (int)isbnList.size(); i++)
    {
        string title = isbnList[i]; // fallback to isbn if book not found
        for (auto &b : Books)
            if (b.get_isbn() == isbnList[i])
            {
                title = b.get_title();
                break;
            }
        cout << title << " (ISBN: " << isbnList[i] << ") - borrowed "
             << countList[i] << " time(s)" << endl;
    }
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//  MEMBER MANAGEMENT
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

void library::registerMember(member m)
{
    for (auto &existing : Members)
    {
        if (existing.get_memberId() == m.get_memberId())
        {
            cout << "Member ID " << m.get_memberId() << " already exists." << endl;
            return;
        }
    }
    Members.push_back(m);
    cout << "Member \"" << m.get_name() << "\" registered successfully." << endl;
}

void library::removeMember(int memberId)
{
    int idx = -1;
    for (int i = 0; i < (int)Members.size(); i++)
        if (Members[i].get_memberId() == memberId)
        {
            idx = i;
            break;
```

```
        }

    if (idx == -1)
    {
        cout << "Member ID " << memberId << " not found." << endl;
        return;
    }

    if (!Members[idx].get_borrowedBooks().empty())
    {
        cout << "Cannot remove member - they still have borrowed books." << endl;
        return;
    }
    cout << "Member \"" << Members[idx].get_name() << "\" removed." << endl;
    Members.erase(Members.begin() + idx);
}

bool library::authenticateMember(int memberId, string contact)
{
    for (auto &m : Members)
        if (m.get_memberId() == memberId && m.get_contactNumber() == contact)
            return true;
    return false;
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//  BORROW & RETURN
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

void library::issueBook(int memberId, string isbn)
{
    member *m = nullptr;
    for (auto &mem : Members)
        if (mem.get_memberId() == memberId)
        {
            m = &mem;
            break;
        }
    if (!m)
    {
        cout << "Member ID " << memberId << " not found." << endl;
        return;
    }

    book *b = nullptr;
    for (auto &bk : Books)
        if (bk.get_isbn() == isbn)
        {
            b = &bk;
            break;
        }
    if (!b)
    {
        cout << "Book with ISBN " << isbn << " not found." << endl;
        return;
    }

    if (b->get_acopies() <= 0)
    {
        cout << "No available copies of \"" << b->get_title() << "\"." << endl;
        return;
    }

    b->set_acopies(b->get_acopies() - 1);
    m->addBorrowedBook(isbn); // isbn stays as string — no stoi

    ofstream fout("transaction.txt", ios::app);
    if (fout.is_open())
    {
        fout << memberId << "|" << isbn << "|today|+14days|0\n";
        fout.close();
    }

    cout << "Book \"" << b->get_title() << "\" issued to \"" << m->get_name() << "\"." << endl;
}

void library::returnBook(int memberId, string isbn)
```

```
{
    member *m = nullptr;
    for (auto &mem : Members)
        if (mem.get_memberId() == memberId)
        {
            m = &mem;
            break;
        }
    if (!m)
    {
        cout << "Member ID " << memberId << " not found." << endl;
        return;
    }

    book *b = nullptr;
    for (auto &bk : Books)
        if (bk.get_isbn() == isbn)
        {
            b = &bk;
            break;
        }
    if (!b)
    {
        cout << "Book with ISBN " << isbn << " not found." << endl;
        return;
    }

    m->removeBorrowedBook(isbn); // isbn stays as string — no stoi
    b->set_acopies(b->get_acopies() + 1);

    // mark transaction as returned (flip trailing 0 to 1)
    ifstream fin("transaction.txt");
    string allContent, line;
    string key = to_string(memberId) + "|" + isbn;
    while (getline(fin, line))
    {
        if (line.find(key) != string::npos && !line.empty() && line.back() == '0')
            line.back() = '1';
        allContent += line + "\n";
    }
    fin.close();

    ofstream fout("transaction.txt");
    fout << allContent;
    fout.close();

    cout << "Book \"" << b->get_title() << "\" returned by \"" << m->get_name() << "\"." << endl;
}

void library::payFine(int memberId)
{
    member *m = nullptr;
    for (auto &mem : Members)
        if (mem.get_memberId() == memberId)
        {
            m = &mem;
            break;
        }
    if (!m)
    {
        cout << "Member ID " << memberId << " not found." << endl;
        return;
    }

    int daysOverdue;
    cout << "Enter number of overdue days for \"" << m->get_name() << "\": ";
    cin >> daysOverdue;

    float fine = m->calculateFine(daysOverdue);
    cout << "Fine for " << daysOverdue << " overdue day(s): Rs." << fine << endl;
    cout << "Fine paid successfully." << endl;
}

// ═══════════════════════════════════════════════
//  PERSISTENCE
// ═══════════════════════════════════════════════
void library::saveBooksToFile()
```

```
{
    ofstream fout("book.txt");
    if (!fout.is_open())
    {
        cout << "book.txt." << endl;
        // cout << "Error opening data/book.txt." << endl;
        return;
    }

    fout << "title|author|genre|isbn|totalCopies|availableCopies\n";
    for (auto &b : Books)
        fout << b.get_title() << "|" << b.get_author() << "|"
             << b.get_genre() << "|" << b.get_isbn() << "|"
             << b.get_tcopies() << "|" << b.get_acopies() << "\n";
    fout.close();
    cout << "Books saved." << endl;
}

void library::loadBooksFromFile()
{
    ifstream fin("book.txt");
    if (!fin.is_open())
    {
        cout << "book.txt not found, starting with empty library." << endl;
        return;
    }

    Books.clear();
    string line;
    getline(fin, line); // skip header

    while (getline(fin, line))
    {
        if (line.empty())
            continue;
        vector<string> f = splitLine(line, '|');
        if (f.size() < 6)
            continue;
        Books.push_back(book(f[0], f[1], f[2], f[3], stoi(f[4]), stoi(f[5])));
    }
    fin.close();
    cout << "Books loaded: " << Books.size() << endl;
}

void library::saveMemberToFile()
{
    ofstream fout("members.txt");
    if (!fout.is_open())
    {
        cout << "members.txt." << endl;
        // cout << "Error opening data/members.txt." << endl;
        return;
    }

    fout << "name|contactNumber|memberId\n";
    for (auto &m : Members)
        fout << m.get_name() << "|" << m.get_contactNumber() << "|" << m.get_memberId() << "\n";
    fout.close();
    cout << "Members saved." << endl;
}

void library::loadMembersFromFile()
{
    ifstream fin("members.txt");
    if (!fin.is_open())
    {
        cout << "members.txt." << endl;
        // cout << "Error opening data/members.txt." << endl;
        return;
    }

    Members.clear();
    string line;
    getline(fin, line); // skip header

    while (getline(fin, line))
    {
```

```
        if (line.empty())
            continue;
        vector<string> f = splitLine(line, '|');
        if (f.size() < 3)
            continue;
        Members.push_back(member(f[0], f[1], stoi(f[2])));
    }
    fin.close();
    cout << "Members loaded: " << Members.size() << endl;
}
```

member.cpp

```
#include <iostream>
#include <vector>
#include "member.h"
#include "user.h"
using namespace std;

member::member() {}

member::member(string pname, string pcontactNumber, int pmemberId)
    : user(pname, pcontactNumber)
{
    memberId = pmemberId;
}

int member::get_memberId() const
{
    return memberId;
}

vector<string> member::get_borrowedBooks() const
{
    return borrowedBooks;
}

void member::set_memberId(int pmemberId)
{
    memberId = pmemberId;
}

void member::addBorrowedBook(string isbn)
{
    borrowedBooks.push_back(isbn);
}

void member::removeBorrowedBook(string isbn)
{
    int idx = -1;
    for (int i = 0; i < (int)borrowedBooks.size(); i++)
    {
        if (borrowedBooks[i] == isbn)
        {
            idx = i;
            break;
        }
    }

    if (idx == -1)
    {
        cout << "ISBN " << isbn << " not found in borrowed list." << endl;
        return;
    }

    borrowedBooks.erase(borrowedBooks.begin() + idx);
}

float member::calculateFine(int pdaysOverDue)
{
    return 2.0f * pdaysOverDue;
}
```

book.cpp

```
#include <iostream>
#include "book.h"

book ::book(string ptitle, string pauthor, string pgenre, string pisbn, int ptcopies, int pacopies)
{
    title = ptitle;
    author = pauthor;
    genre = pgenre;
    isbn = pisbn;
    totalCopies = ptcopies;
    availableCopies = pacopies;
}

string book ::get_title() const
{
    return title;
}
string book ::get_author() const
{
    return author;
}
string book ::get_genre() const
{
    return genre;
}
string book ::get_isbn() const
{
    return isbn;
}

int book ::get_tcopies() const
{
    return totalCopies;
}
int book ::get_acopies() const
{
    return availableCopies;
}

void book ::set_title(string ptitle)
{
    title = ptitle;
}
void book ::set_author(string pauthor)
{
    author = pauthor;
}
void book ::set_genre(string pgenre)
{
    genre = pgenre;
}
void book ::set_isbn(string pisbn)
{
    isbn = pisbn;
}
void book ::set_tcopies(int ptcopies)
{
    totalCopies = ptcopies;
}
void book ::set_acopies(int pacopies)
{
    availableCopies = pacopies;
}

ostream& operator<<(ostream& os, const book& b) {
    os << b.title << " | " << b.author << " | " << b.genre
       << " | " << b.isbn << " | " << b.availableCopies << "/" << b.totalCopies;
    return os;
}
```

main.cpp

```
#include <iostream>
#include <limits>
#include "library.h"
#include "book.h"
#include "member.h"
#include "admin.h"
using namespace std;

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//  HELPERS
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

void clearScreen() { cout << "\033[2J\033[1;1H"; }

void pause()
{
    cout << "\nPress Enter to continue...";
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cin.get();
}

void printHeader(const string &title)
{
    cout << "\n■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■\n";
    cout << "■  " << title;
    int pad = 32 - (int)title.size();
    for (int i = 0; i < pad; i++)
        cout << ' ';
    cout << "■\n";
    cout << "■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■\n";
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//  ADMIN MENUS
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

void adminBookMenu(library &lib)
{
    int choice;
    do
    {
        clearScreen();
        printHeader("BOOK MANAGEMENT [ADMIN]");
        cout << "1. Add Book\n";
        cout << "2. Remove Book\n";
        cout << "3. Update Book\n";
        cout << "4. Search Book\n";
        cout << "5. Display All Books\n";
        cout << "6. Display Available Books\n";
        cout << "7. Display Borrowed Books\n";
        cout << "8. Display Overdue Books\n";
        cout << "9. Most Borrowed Books\n";
        cout << "0. Back\n";
        cout << "Choice: ";
        cin >> choice;

        if (choice == 1)
        {
            string title, author, genre, isbn;
            int tc, ac;
            cin.ignore();
            cout << "Title: ";
            getline(cin, title);
            cout << "Author: ";
            getline(cin, author);
            cout << "Genre: ";
            getline(cin, genre);
            cout << "ISBN: ";
            getline(cin, isbn);
            cout << "Total copies: ";
            cin >> tc;
            cout << "Available copies: ";
            cin >> ac;
```

```
            lib.addBook(book(title, author, genre, isbn, tc, ac));
        }
        else if (choice == 2)
        {
            string isbn;
            cout << "ISBN: ";
            cin >> isbn;
            lib.removeBook(isbn);
        }
        else if (choice == 3)
        {
            string isbn;
            cout << "ISBN: ";
            cin >> isbn;
            lib.updateBook(isbn);
        }
        else if (choice == 4)
        {
            string q;
            cin.ignore();
            cout << "Search: ";
            getline(cin, q);
            lib.searchBook(q);
        }
        else if (choice == 5)
            lib.displayAllBooks();
        else if (choice == 6)
            lib.displayAvailableBooks();
        else if (choice == 7)
            lib.displayBorrowedBooks();
        else if (choice == 8)
            lib.displayOverdueBooks();
        else if (choice == 9)
            lib.getMostBorrowedBooks();

        if (choice != 0)
            pause();

    } while (choice != 0);
}

void adminMemberMenu(library &lib)
{
    int choice;
    do
    {
        clearScreen();
        printHeader("MEMBER MANAGEMENT [ADMIN]");
        cout << "1. Register Member\n";
        cout << "2. Remove Member\n";
        cout << "0. Back\n";
        cout << "Choice: ";
        cin >> choice;

        if (choice == 1)
        {
            string name, contact;
            int id;
            cin.ignore();
            cout << "Name: ";
            getline(cin, name);
            cout << "Contact Number: ";
            getline(cin, contact);
            cout << "Member ID: ";
            cin >> id;
            lib.registerMember(member(name, contact, id));
        }
        else if (choice == 2)
        {
            int id;
            cout << "Member ID: ";
            cin >> id;
            lib.removeMember(id);
        }

        if (choice != 0)
            pause();
```

```
    } while (choice != 0);
}

void adminBorrowMenu(library &lib)
{
    int choice;
    do
    {
        clearScreen();
        printHeader("BORROW & RETURN [ADMIN]");
        cout << "1. Issue Book to Member\n";
        cout << "2. Return Book from Member\n";
        cout << "3. Collect Fine\n";
        cout << "0. Back\n";
        cout << "Choice: ";
        cin >> choice;

        if (choice == 1)
        {
            int id;
            string isbn;
            cout << "Member ID: ";
            cin >> id;
            cout << "ISBN: ";
            cin >> isbn;
            lib.issueBook(id, isbn);
        }
        else if (choice == 2)
        {
            int id;
            string isbn;
            cout << "Member ID: ";
            cin >> id;
            cout << "ISBN: ";
            cin >> isbn;
            lib.returnBook(id, isbn);
        }
        else if (choice == 3)
        {
            int id;
            cout << "Member ID: ";
            cin >> id;
            lib.payFine(id);
        }

        if (choice != 0)
            pause();

    } while (choice != 0);
}

void adminDashboard(library &lib)
{
    int choice;
    do
    {
        clearScreen();
        printHeader("ADMIN DASHBOARD");
        cout << "1. Book Management\n";
        cout << "2. Member Management\n";
        cout << "3. Borrow & Return\n";
        cout << "4. Save Data\n";
        cout << "0. Logout\n";
        cout << "Choice: ";
        cin >> choice;

        if (choice == 1)
            adminBookMenu(lib);
        else if (choice == 2)
            adminMemberMenu(lib);
        else if (choice == 3)
            adminBorrowMenu(lib);
        else if (choice == 4)
        {
            lib.saveBooksToFile();
            lib.saveMemberToFile();
            pause();
```

```
        }
        else if (choice == 0)
        {
            lib.saveBooksToFile();
            lib.saveMemberToFile();
            cout << "Data saved. Logged out.\n";
        }

    } while (choice != 0);
}

// ═══════════════════════════════════════════════
//  MEMBER MENU
// ═══════════════════════════════════════════════

void memberDashboard(library &lib, int memberId)
{
    int choice;
    do
    {
        clearScreen();
        printHeader("MEMBER DASHBOARD");
        cout << "1. Search Book\n";
        cout << "2. View Available Books\n";
        cout << "3. Borrow a Book\n";
        cout << "4. Return a Book\n";
        cout << "5. Pay Fine\n";
        cout << "0. Logout\n";
        cout << "Choice: ";
        cin >> choice;

        if (choice == 1)
        {
            string q;
            cin.ignore();
            cout << "Search (title/author): ";
            getline(cin, q);
            lib.searchBook(q);
        }
        else if (choice == 2)
            lib.displayAvailableBooks();
        else if (choice == 3)
        {
            string isbn;
            cout << "Enter ISBN of book to borrow: ";
            cin >> isbn;
            lib.issueBook(memberId, isbn);
        }
        else if (choice == 4)
        {
            string isbn;
            cout << "Enter ISBN of book to return: ";
            cin >> isbn;
            lib.returnBook(memberId, isbn);
        }
        else if (choice == 5)
            lib.payFine(memberId);
        else if (choice == 0)
            cout << "Logged out.\n";

        if (choice != 0)
            pause();

    } while (choice != 0);
}

// ═══════════════════════════════════════════════
//  LOGIN
// ═══════════════════════════════════════════════

// Hardcoded admins — you can extend this to load from file
admin admins[] = {
    admin("Super Admin", "0000000000", 1, "admin123"),
    admin("Librarian", "1111111111", 2, "lib456")};
int adminCount = 2;

void loginScreen(library &lib)
```

```
{
    int choice;
    do
    {
        clearScreen();
        printHeader("LIBRARY MANAGEMENT SYSTEM");
        cout << "1. Login as Admin\n";
        cout << "2. Login as Member\n";
        cout << "0. Exit\n";
        cout << "Choice: ";
        cin >> choice;

        // ■■ ADMIN LOGIN ■■
        if (choice == 1)
        {
            int adminId;
            string password;
            cout << "\nAdmin ID: ";
            cin >> adminId;
            cout << "Password: ";
            cin >> password;

            bool found = false;
            for (int i = 0; i < adminCount; i++)
            {
                if (admins[i].get_adminId() == adminId && admins[i].authenticate(password))
                {
                    cout << "\nWelcome, " << admins[i].get_name() << "!\n";
                    pause();
                    adminDashboard(lib);
                    found = true;
                    break;
                }
            }
            if (!found)
            {
                cout << "\nInvalid Admin ID or Password.\n";
                pause();
            }
        }

        // ■■ MEMBER LOGIN ■■
        else if (choice == 2)
        {
            int memberId;
            string contact;
            cout << "\nMember ID: ";
            cin >> memberId;
            cout << "Contact Number: ";
            cin >> contact;

            bool found = lib.authenticateMember(memberId, contact);
            if (found)
            {
                cout << "\nWelcome!\n";
                pause();
                memberDashboard(lib, memberId);
            }
            else
            {
                cout << "\nInvalid Member ID or Contact Number.\n";
                pause();
            }
        }

        else if (choice == 0)
        {
            lib.saveBooksToFile();
            lib.saveMemberToFile();
            cout << "Goodbye!\n";
        }

    } while (choice != 0);
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//  MAIN
```

```
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

int main()
{
    library lib;
    lib.loadBooksFromFile();
    lib.loadMembersFromFile();

    loginScreen(lib);
    return 0;
}
```